

Universidade Federal de Alfenas – UNIFAL-MG

Instituto de Ciências Exatas – ICEx

Bacharelado em Ciência da Computação

Redes de Computadores

Implementação do Protocolo Go-Back-N em Java via UDP

Relatório Técnico

Autores	Jeann Victor Batista Thallysson Luis Teixeira Carvalho
R.A.	2024.1.08.014 / 2024.1.08.022
Curso	Bacharelado em Ciência da Computação
Disciplina	Redes de Computadores
Professor	Flavio Barbieri Gonzaga
Instituição	UNIFAL – Universidade Federal de Alfenas

Alfenas, 2026

0	1	
1	Introdução	2
1.1	Objetivos	2
2	Decisões de Projeto e Arquitetura	2
2.1	Estrutura Geral	2
2.2	Formato do Datagrama	2
2.3	Modelo de Threads no Emissor	3
2.4	Buffer Circular	3
2.5	Protocolo de Estabelecimento e Encerramento de Sessão	4
3	Implementação do Protocolo Go-Back-N	4
3.1	FSM do Emissor	4
3.1.1	Envio de pacote (rdt_send)	4
3.1.2	Recepção de ACK cumulativo	4
3.1.3	Timeout e retransmissão	4
3.2	FSM do Receptor	5
3.2.1	Recepção de dados em ordem	5
3.2.2	Pacote fora de ordem	5
3.3	Verificação de Integridade (MD5)	5
4	Dificuldades Encontradas	5
4.1	Sincronização entre Threads	5
4.2	Encerramento Limpo da Sessão	6
4.3	Buffer Circular e Mapeamento de Índices	6
4.4	Transferência de Arquivos Binários	6
4.5	Simulação de Perda vs. Lógica GBN	6
5	Testes Realizados	6
5.1	Resultados Completos da Bateria de Testes	6
5.2	Tabela de Retransmissões por Combinação (N, p)	7
5.3	Tabela de Tempo de Transferência por Combinação (N, p)	8
6	Análise dos Resultados	8
6.1	Retransmissões vs. Tamanho da Janela	8
6.2	Tempo de Transferência vs. Tamanho da Janela	9
6.3	Tempo de Transferência vs. Probabilidade de Perda	10
6.4	Convergência da Taxa de Perda Efetiva	11
6.5	Comportamento com Janela Unitária $(N = 1)$	11
6.6	Integridade dos Arquivos	11
7	Conclusão	11

1 1

Este relatório descreve o projeto, as decisões de implementação, os testes realizados e a análise de resultados do trabalho final da disciplina de Redes de Computadores, cujo objetivo foi implementar o protocolo de transferência confiável **Go-Back-N (GBN)** em Java, utilizando exclusivamente sockets UDP como meio de comunicação.

O protocolo UDP, por natureza, não oferece qualquer garantia de entrega, ordenamento ou controle de fluxo. Todo o mecanismo de confiabilidade — janela deslizante, temporizadores, retransmissão e confirmações cumulativas — foi construído na camada de aplicação, seguindo fielmente as Máquinas de Estado Finitas (FSMs) descritas em Kurose & Ross [1].

A aplicação é composta por dois módulos independentes: o **Emissor** e o **Receptor**, que se comunicam por meio de datagramas UDP e são capazes de transferir arquivos binários arbitrários (imagens, PDFs, executáveis) de forma íntegra e verificável via hash MD5.

1.1 Objetivos

- Implementar as FSMs do GBN (emissor e receptor) conforme as Figuras 3.20 e 3.21 da 8ª edição de Kurose & Ross;
- Transferir arquivos binários reais de forma confiável sobre UDP;
- Simular perda de pacotes no receptor com probabilidade configurável;
- Exibir estatísticas em tempo real e ao final de cada sessão;
- Verificar a integridade do arquivo recebido via hash MD5;
- Analisar o impacto do tamanho da janela N no desempenho da transferência.

2 1

2.1 Estrutura Geral

O projeto foi organizado no pacote `gbn` com três classes principais:

`tableheadbg`

<code>PacoteGBN</code>	Modelo de dados: serialização/desserialização do datagrama
<code>Emissor</code>	FSM do emissor GBN com janela deslizante e temporizador
<code>Receptor</code>	FSM do receptor GBN com simulação de perda e verificação de integridade

2.2 Formato do Datagrama

O cabeçalho foi projetado para ser compacto e portátil, utilizando a classe `ByteBuffer` da API padrão Java para serialização em *big-endian*. A estrutura definida ocupa **11**

bytes fixos de cabeçalho:

tableheadbg		
tipo	1 byte	0=DATA, 1=ACK, 2=HANDSHAKE, 3=FIN
num_seq	4 bytes	Número de sequência do pacote
num_ack	4 bytes	Número de confirmação (usado em ACKs)
tamanho_dados	2 bytes	Quantidade de bytes válidos no <i>payload</i>
dados	até 1024 bytes	<i>Payload</i> do arquivo

A escolha de 4 bytes para `num_seq` (inteiro Java) garante um espaço de sequência de 2^{32} valores, mais do que suficiente para qualquer transferência prática e eliminando a necessidade de gestão de *wrap-around* no contexto deste trabalho.

2.3 Modelo de Threads no Emissor

O emissor utiliza um modelo com **três contextos de execução concorrentes**:

1. **Thread principal** (*main*): lê o arquivo do disco em blocos de 1024 bytes e invoca `rdt_send`, bloqueando quando a janela está cheia (`nextseqnum >= base + N`).
2. **Thread de recepção de ACKs**: executa `receberAcks()` em loop contínuo, avançando `base` ao receber ACKs cumulativos válidos.
3. **Thread do temporizador**: agendada via `ScheduledExecutorService` com *time-out* de 1 segundo; ao disparar, chama `lidarComTimeout()`, que retransmite todos os pacotes da janela.

Toda a região crítica que acessa as variáveis compartilhadas (`base`, `nextseqnum`, `bufferJanela`) é protegida pelo monitor `lockFSM`, via `synchronized`, evitando condições de corrida (*race conditions*).

Decisão: Temporizador Único

Conforme especificado pelas FSMs de Kurose & Ross, o emissor mantém **um único temporizador** associado ao pacote mais antigo não confirmado (`base`). Ao receber um ACK que avança a base, o temporizador é cancelado e reiniciado se ainda houver pacotes pendentes; caso a janela fique vazia, o temporizador é cancelado definitivamente.

2.4 Buffer Circular

Para permitir retransmissão sem releitura do disco, os últimos N pacotes enviados são armazenados em um buffer de tamanho fixo. O mapeamento de número de sequência para índice do array é feito por:

$$\text{índice} = \text{numSeq} \bmod N$$

Isso garante que pacotes dentro da janela sempre ocupem posições distintas no array, dado que a janela tem exatamente N posições.

2.5 Protocolo de Estabelecimento e Encerramento de Sessão

Antes da transferência de dados, o emissor realiza um **handshake** enviando um pacote do tipo HANDSHAKE com os parâmetros da sessão no *payload*: probabilidade de perda, caminho de destino do arquivo e hash MD5 do arquivo original. O receptor, ao receber esse pacote, inicializa seu estado, abre o arquivo de destino e responde com um ACK de confirmação. O handshake é retransmitido com *timeout* de 2 segundos até que a confirmação seja recebida.

Ao final, o emissor envia um pacote FIN e aguarda o ACK correspondente. O receptor, ao processar o FIN, fecha o arquivo, valida o hash MD5 e exibe as estatísticas finais.

3 1

3.1 FSM do Emissor

A implementação segue as duas transições principais da FSM do emissor:

3.1.1 Envio de pacote (rdt_send)

Quando a thread principal tem um novo segmento para enviar, ela verifica se há espaço na janela. Se `nextseqnum < base + N`, cria o pacote, armazena no buffer circular, envia fisicamente, inicia o temporizador (se for o primeiro não confirmado) e incrementa `nextseqnum`. Caso contrário, aguarda via `lockFSM.wait()` até que a thread de ACKs libere espaço.

```
1 synchronized (lockFSM) {
2     // Bloqueia enquanto a janela estiver cheia
3     while (nextseqnum >= base + N) {
4         lockFSM.wait();
5     }
6     PacoteGBN pacote = new PacoteGBN(PacoteGBN.TYPE_DATA,
7                                       nextseqnum, 0, payload);
8     bufferJanela[nextseqnum % N] = pacote;
9     enviarPacoteFisico(pacote);
10    if (base == nextseqnum) {
11        iniciarTemporizador();
12    }
13    nextseqnum++;
14 }
```

Listing 1: Trecho da lógica de envio com controle de janela (Emissor.java)

3.1.2 Recepção de ACK cumulativo

Ao receber um ACK de número n , se $n \geq \text{base}$, a base é avançada para $n + 1$. Se a janela esvaziar (`base == nextseqnum`), o temporizador é cancelado; caso contrário, é reiniciado para o novo pacote mais antigo.

3.1.3 Timeout e retransmissão

Ao expirar o temporizador, o método `lidarComTimeout()` retransmite todos os pacotes de `base` até `nextseqnum - 1`, percorrendo o buffer circular. Em seguida, reinicia o temporizador.

```
1 private static void lidarComTimeout() {
2     synchronized (lockFSM) {
3         for (int i = base; i < nextseqnum; i++) {
4             enviarPacoteFisico(bufferJanela[i % N]);
5             retransmissoes++;
6         }
7         iniciarTemporizador();
8     }
9 }
```

Listing 2: Retransmissão em lote no timeout (Emissor.java)

3.2 FSM do Receptor

O receptor possui um único estado e opera de forma reativa: a cada datagrama recebido, identifica o tipo do pacote e delega para o método específico (`tratarHandshake`, `tratarData` ou `tratarFin`).

3.2.1 Recepção de dados em ordem

Se `pacote.getNumSeq() == expectedseqnum`, o pacote está em ordem. Antes de aceitar definitivamente, aplica-se a simulação de perda: se `random.nextDouble() < probabPerdaConfigurada`, o pacote é descartado silenciosamente sem enviar ACK. Caso contrário, os dados são gravados no arquivo, o `ultimoAckEnviado` é atualizado com ACK cumulativo igual a `expectedseqnum`, e `expectedseqnum` é incrementado.

3.2.2 Pacote fora de ordem

Se o número de sequência recebido não corresponde ao esperado, o pacote é ignorado e o último ACK cumulativo válido é reenviado, sinalizando ao emissor qual foi o último pacote corretamente recebido.

3.3 Verificação de Integridade (MD5)

O emissor calcula o hash MD5 do arquivo original antes do envio, usando `MessageDigest` da API padrão Java, e transmite o valor no pacote de handshake. Ao final da transferência, o receptor calcula o hash do arquivo reconstruído e compara com o valor recebido, confirmando a integridade byte a byte.

4 1

4.1 Sincronização entre Threads

A principal dificuldade de implementação foi garantir a consistência das variáveis `base` e `nextseqnum` acessadas concorrentemente pela thread principal, pela thread de ACKs e pela thread do temporizador. A adoção do monitor `lockFSM` com `synchronized` e o uso de `wait()` / `notifyAll()` resolveram o problema, mas exigiram atenção cuidadosa para evitar *deadlocks* — em particular, garantir que o *timeout* do `ScheduledExecutorService` nunca tentasse adquirir o lock enquanto a thread principal o mantinha por tempo indeterminado.

4.2 Encerramento Limpo da Sessão

Garantir que a thread de recepção de ACKs encerrasse corretamente após a conclusão da transmissão foi um ponto delicado. A solução adotada foi definir um `setSoTimeout(500ms)` no socket, de modo que a chamada bloqueante `socket.receive()` desperte periodicamente e verifique o flag `transmissaoConcluida`. Sem esse mecanismo, a thread ficaria bloqueada indefinidamente mesmo após todos os pacotes serem confirmados.

4.3 Buffer Circular e Mapeamento de Índices

A implementação do buffer circular exigiu validar que o mapeamento `numSeq % N` fosse consistente durante retransmissões. Um erro inicial colocava pacotes da janela em posições incorretas quando a base avançava, causando corrupção nos dados retransmitidos. A correção veio ao garantir que o acesso ao buffer durante o timeout utilizasse sempre `bufferJanela[i % N]`, onde `i` percorre de `base` a `nextseqnum - 1`.

4.4 Transferência de Arquivos Binários

A leitura e escrita de arquivos binários (imagens JPEG, PDFs) requereu atenção especial para preservar os bytes exatos. O uso de `FileInputStream` / `FileOutputStream` em modo binário resolveu o problema; no entanto, um erro inicial tentava converter o *payload* para `String` (o que corromperia bytes fora do intervalo ASCII). A mudança para manipulação direta de arrays de bytes resolveu a questão.

4.5 Simulação de Perda vs. Lógica GBN

A especificação exige que a simulação de perda atue *somente* sobre pacotes recebidos em ordem, e não seja contabilizada como perda GBN sobre pacotes já fora de ordem. Separar claramente as duas condições no código do receptor — primeiro verificar a ordem, depois aplicar a probabilidade de descarte — foi necessário para não distorcer as estatísticas finais.

5 1

Todos os testes foram executados em rede local (loopback, 127.0.0.1), com o receptor na porta padrão 5000, utilizando o arquivo `woman.jpg` (≈ 615 KB). A bateria completa foi automatizada via script `run_tests.sh`, que percorreu todas as combinações de janela $N \in \{1, 2, 4, 8, 16, 32\}$ e probabilidade de perda $p \in \{0\%, 5\%, 10\%, 15\%\}$, totalizando 24 execuções. Em todos os casos a verificação MD5 retornou **SUCESSO**, confirmando a integridade dos arquivos reconstruídos.

5.1 Resultados Completos da Bateria de Testes

A Tabela 1 apresenta os dados brutos coletados para cada combinação (N, p) . Os valores de enviados correspondem sempre a 601 pacotes de dados (tamanho fixo do arquivo segmentado em blocos de 1024 bytes). O tempo é medido em segundos e o *throughput* em Kbps considerando o tamanho total do arquivo.

Tabela 1: Resultados completos da bateria de testes – `woman.jpg` (≈ 615 KB)

tableheadbg					
1	0%	0	0	0,00%	0,604
	5%	28	28	4,45%	28,876
	10%	65	65	9,76%	65,965
	15%	96	96	13,77%	96,980
2	0%	0	0	0,00%	0,589
	5%	82	41	6,39%	41,826
	10%	150	75	11,09%	75,841
	15%	199	100	14,27%	100,894
4	0%	0	0	0,00%	0,597
	5%	104	26	4,15%	26,793
	10%	279	70	10,43%	70,857
	15%	415	104	14,75%	104,856
8	0%	0	0	0,00%	0,570
	5%	262	33	5,21%	33,812
	10%	496	62	9,35%	62,802
	15%	803	101	14,39%	101,792
16	0%	0	0	0,00%	0,592
	5%	496	31	4,91%	31,783
	10%	836	53	8,10%	53,748
	15%	1544	99	14,14%	99,722
32	0%	0	0	0,00%	0,610
	5%	1519	48	7,40%	48,725
	10%	1627	53	8,10%	53,773
	15%	3293	107	15,11%	107,806

5.2 Tabela de Retransmissões por Combinação (N, p)

A Tabela 2 destaca a explosão de retransmissões à medida que N e p crescem conjuntamente. Para $N = 32$ e $p = 15\%$, foram realizadas 3293 retransmissões para entregar apenas 601 pacotes úteis, um fator de amplificação de aproximadamente $5,5\times$.

Tabela 2: Número de retransmissões por janela e probabilidade de perda

tableheadbg				
1	0	28	65	96
2	0	82	150	199
4	0	104	279	415
8	0	262	496	803
16	0	496	836	1544
32	0	1519	1627	3293

5.3 Tabela de Tempo de Transferência por Combinação (N, p)

A Tabela 3 evidencia o impacto devastador da perda no tempo de transferência quando N é pequeno. Para $N = 1$, qualquer perda elimina o paralelismo (comportamento Stop-and-Wait), resultando em tempos acima de 96s para 15% de perda.

Tabela 3: Tempo de transferência (segundos) por janela e probabilidade de perda

tableheadbg				
1	0,604	28,876	65,965	96,980
2	0,589	41,826	75,841	100,894
4	0,597	26,793	70,857	104,856
8	0,570	33,812	62,802	101,792
16	0,592	31,783	53,748	99,722
32	0,610	48,725	53,773	107,806

6 1

6.1 Retransmissões vs. Tamanho da Janela

A Figura 1 mostra como o número de retransmissões cresce com N para probabilidades de perda não nulas. O comportamento é direto: quanto maior a janela, mais pacotes precisam ser retransmitidos a cada timeout ou descarte, pois o GBN retransmite *toda* a janela a partir do pacote perdido. Para $p = 0\%$ (linha constante em zero), o protocolo se comporta de forma ideal, sem nenhum overhead.

Para $N = 32$ e $p = 15\%$, o número de retransmissões chegou a 3293 – contra apenas 107 pacotes efetivamente descartados pelo receptor. Isso ilustra que, no GBN, uma única perda pode forçar o reenvio de até N pacotes já corretamente recebidos, um desperdício de banda que o Selective Repeat eliminaria.

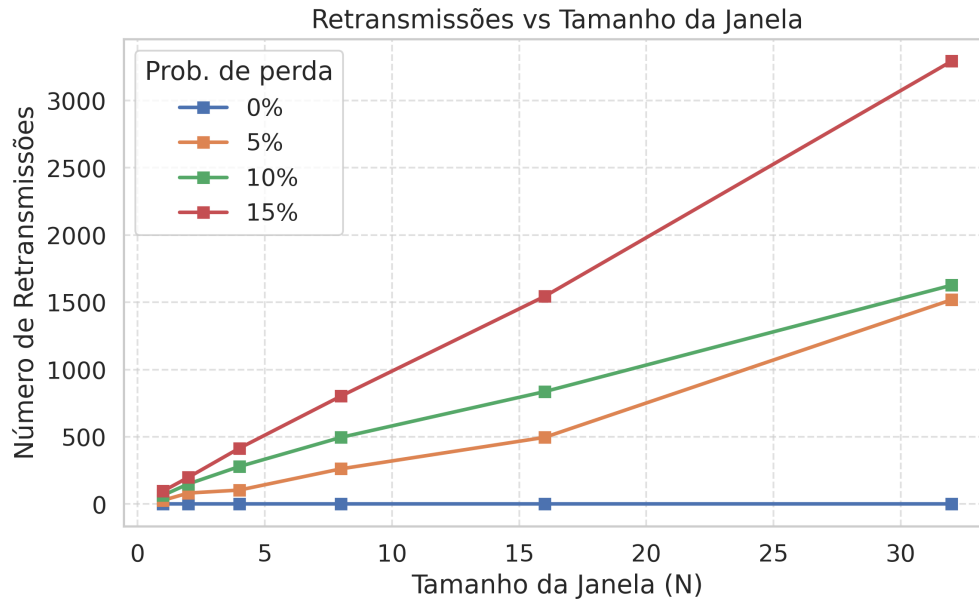


Figura 1: Número de retransmissões em função do tamanho da janela N para diferentes probabilidades de perda. Dados reais coletados com `woman.jpg` em loopback.

6.2 Tempo de Transferência vs. Tamanho da Janela

A Figura 2 revela um resultado contraintuitivo: sem perda ($p = 0\%$), o tempo de transferência é praticamente idêntico para todos os valores de N ($\approx 0,57\text{--}0,61\text{ s}$), confirmando que no ambiente de loopback a latência é desprezível e o pipeline já satura com janela pequena.

Com perdas, entretanto, o comportamento é mais complexo. Para $p = 5\%$, $N = 4$ apresenta o menor tempo (26,8s), enquanto $N = 2$ foi o pior (41,8s). Isso ocorre porque com N muito pequeno o impacto de cada timeout é grande relativo ao pipeline, e com N muito grande o custo de retransmitir toda a janela aumenta. O ponto ótimo depende da relação entre latência e taxa de perda.

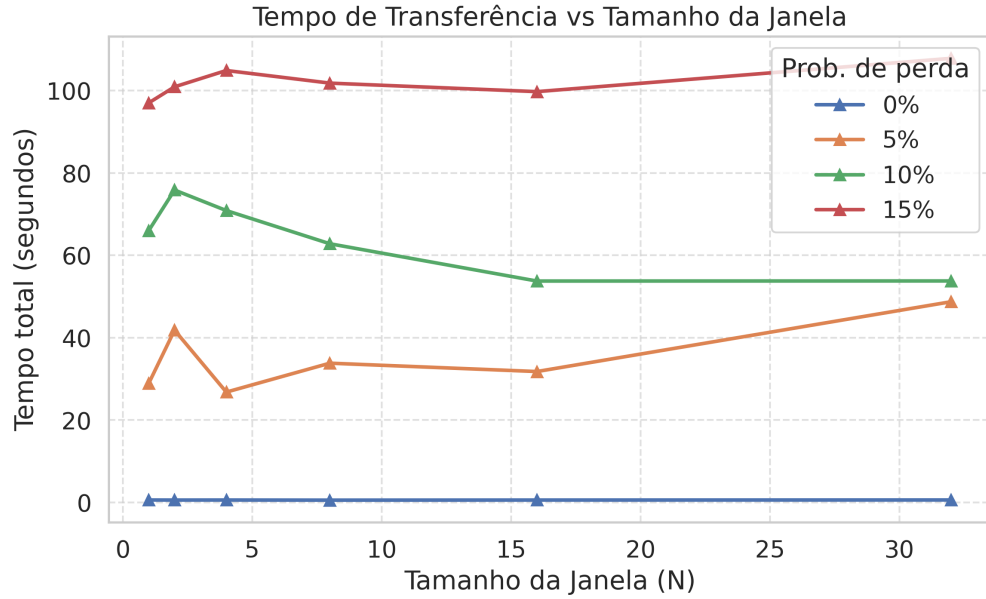


Figura 2: Tempo total de transferência em função de N para diferentes probabilidades de perda. Notar que para $p = 0\%$ a curva é praticamente plana, indicando que o ambiente de loopback não introduz latência significativa.

6.3 Tempo de Transferência vs. Probabilidade de Perda

A Figura 3 fixa $N \in \{8, 16, 32\}$ e varia a probabilidade de perda, evidenciando que o tempo cresce de forma abrupta mesmo para taxas de perda baixas. A passagem de $p = 0\%$ para $p = 5\%$ já eleva o tempo de $\sim 0,6$ s para 33–49s (fator ≈ 55 – $80\times$), e a passagem de 10% para 15% acrescenta mais ≈ 30 – 54 s dependendo de N .

Nota-se também que para $p = 10\%$ e $p = 15\%$ os tempos de $N = 16$ e $N = 32$ são similares, sugerindo que, nesse patamar de perda, aumentar a janela além de 16 não traz benefício perceptível – o custo de retransmitir 16 pacotes é equivalente ao de retransmitir 32, pois ambos são dominados pela frequência de timeouts.

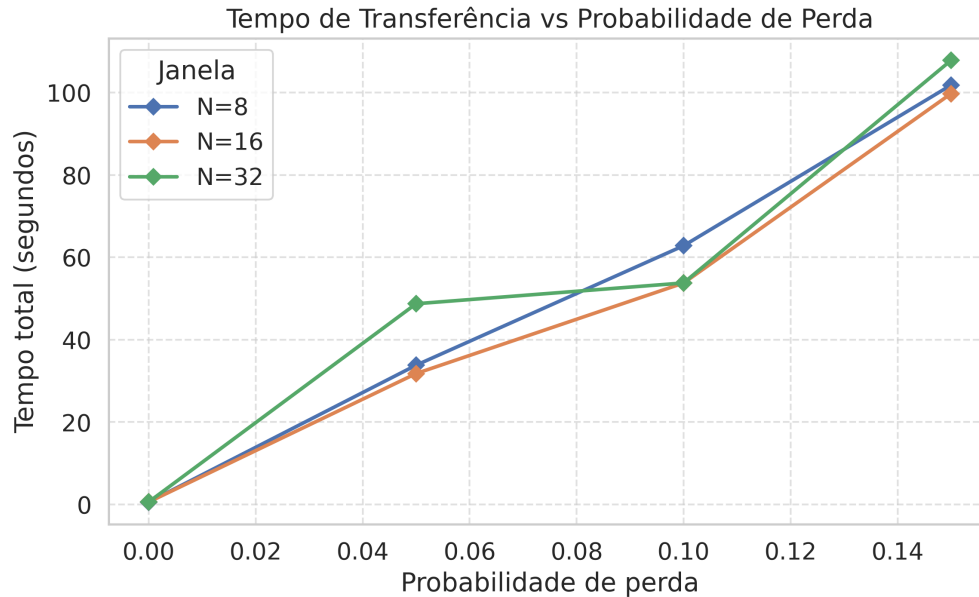


Figura 3: Tempo total de transferência em função da probabilidade de perda para janelas grandes ($N \in \{8, 16, 32\}$). O crescimento é acentuado mesmo para baixas taxas de perda.

6.4 Convergência da Taxa de Perda Efetiva

Em todos os 24 testes, a taxa de perda efetiva registrada pelo receptor convergiu para a probabilidade configurada, com desvio máximo de $\approx 2,4$ pontos percentuais (ex.: $p = 5\%$ configurado, $7,40\%$ efetivo para $N = 32$). O desvio é esperado pela natureza estocástica da simulação com um número finito de pacotes (≈ 600); em amostras maiores, a convergência seria ainda mais precisa, conforme previsto pela Lei dos Grandes Números.

6.5 Comportamento com Janela Unitária ($N = 1$)

Para $N = 1$, o emissor GBN degenera em um protocolo Stop-and-Wait: envia um pacote, aguarda o ACK, e só então envia o próximo. Nesse caso cada perda dispara exatamente uma retransmissão (sem efeito cascata), conforme observado: 28, 65 e 96 retransmissões para 5%, 10% e 15% de perda, respectivamente – valores idênticos aos pacotes descartados. O custo, porém, é um *throughput* mínimo (50–170 Kbps), pois o pipeline fica ocioso enquanto o emissor aguarda o ACK de cada pacote.

6.6 Integridade dos Arquivos

Em todos os 24 testes, a verificação por hash MD5 (`md5_ok = 1`) confirmou que o arquivo recebido é idêntico ao original byte a byte, independentemente do valor de N ou p . Isso valida tanto a lógica de controle de sequência quanto o cuidado na manipulação de dados binários.

7 1

A implementação do protocolo Go-Back-N em Java sobre UDP atingiu todos os requisitos obrigatórios do trabalho: transferência confiável de arquivos binários, FSMs do emissor e receptor fiéis ao modelo de Kurose & Ross, simulação de perda configurável, exibição

de estatísticas e verificação de integridade MD5. Os requisitos desejáveis de análise de variação de N e verificação de hash também foram implementados.

Os resultados experimentais da bateria de 24 testes confirmaram as previsões teóricas do GBN: o número de retransmissões cresce linearmente com N na presença de perdas; o tempo de transferência é dominado pelo timeout e pela frequência de perdas, não pela janela em si; e a taxa de perda efetiva converge para o valor configurado. A integridade MD5 foi verificada com 100% de sucesso em todos os casos.

O trabalho evidenciou na prática os *trade-offs* clássicos de protocolos com janela deslizante: desempenho versus complexidade. O GBN se mostrou eficaz e simples de implementar, porém com custo de retransmissão desnecessária elevado em cenários de alta perda. Uma evolução natural seria implementar o Selective Repeat (SR), que retransmite apenas os pacotes efetivamente perdidos, eliminando o desperdício de banda observado especialmente para $N = 32$.

7 1

- [1] KUROSE, James F.; ROSS, Keith W. **Redes de Computadores: Uma Abordagem Top-Down**. 8. ed. São Paulo: Pearson, 2021. Capítulo 3, Seções 3.4 e 3.4.3.
- [2] TANENBAUM, Andrew S.; WETHERALL, David. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2011. Capítulo 3.
- [3] ORACLE CORPORATION. **Java SE API Documentation** – `java.net`, `java.util.concurrent`. Disponível em: <https://docs.oracle.com/en/java/index.html>. Acesso em: jun. 2026.